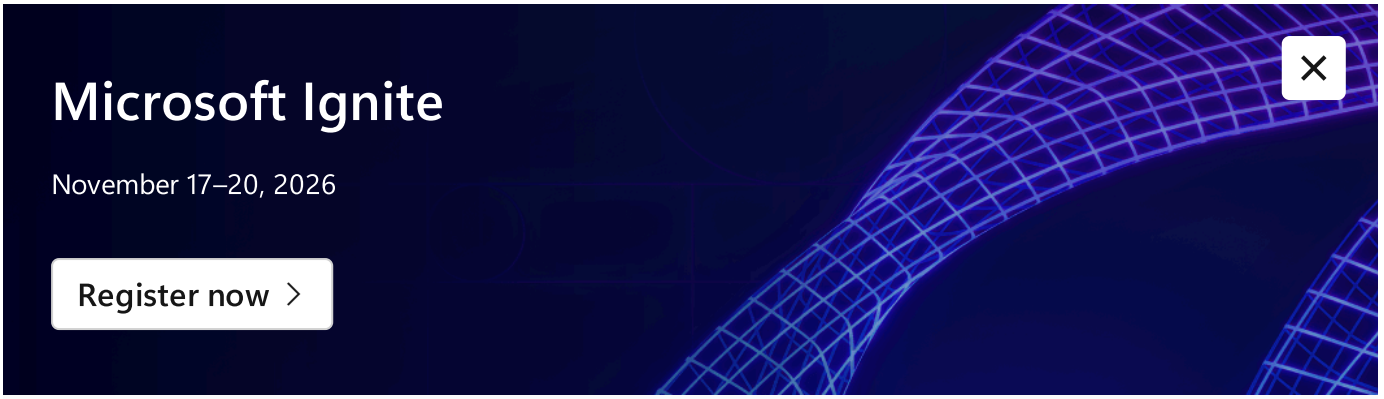

Learn

Sign in

Architecture best practices for Azure Databricks

Summarize this article for me

Azure Databricks is an Apache Spark-based analytics platform that data teams can use to collaborate efficiently. Use it to build and deploy data engineering, machine learning, and analytics workloads at scale. This article covers common considerations and best practices for Azure Databricks and gives architectural recommendations mapped to the Azure Well-Architected Framework pillars.

It's assumed that as an architect, you've reviewed Choose an analytical data store and chose Azure Databricks as the analytics platform for your workload.

Technology scope

This review focuses on the interrelated decisions for the following Azure resources:

- Azure Databricks
- Spark
- Delta Lake
- Unity Catalog
- MLflow

Reliability

The purpose of the Reliability pillar is to provide continued functionality by **building enough resilience and the ability to recover fast from failures**.

Reliability design principles provide a high-level design strategy applied for individual components, system flows, and the system as a whole.

Workload design checklist

Start your design strategy based on the design review checklist for Reliability. Determine its relevance to your business requirements while keeping in mind the nature of your application and the criticality of its components. Extend the strategy to include more approaches as needed.

- ✓ **Understand service limits and quotas.** Azure Databricks service limits directly constrain workload reliability through compute clusters, workspace capacity, storage throughput, and network bandwidth restrictions. Your architecture design must proactively incorporate quotas to prevent unexpected service disruptions that can halt scaling operations during peak demand. These quotas include a 1000-node cluster limit, workspace cluster maximums, and regional capacity constraints.

- ✓ **Use failure mode analysis (FMA) to anticipate potential failures.** Systematic FMA identifies potential system failures and establishes corresponding mitigation strategies to maintain distributed computing resilience.

  The following table includes common failure scenarios and their proven mitigation approaches.

Expand table

Failure	Mitigation
Cluster driver node failure	Use cluster automatic-restart policies and implement checkpointing for Spark applications. Use structured streaming with fault-tolerant state management.
Job execution failures	Implement retry policies with exponential backoff. Use Azure Databricks job orchestration with error handling. Set up appropriate timeout settings.
Data corruption or inconsistency	Use Delta Lake atomicity, consistency, isolation, and durability (ACID) transactions, time travel capabilities, and data expectations in Lakeflow Spark Declarative Pipelines. Implement data validation checks and monitoring.
Workspace or region unavailability	Implement multiregion deployment strategies. Use workspace backup and restore procedures. Set up cross-region data replication.

These mitigation strategies use native Azure Databricks capabilities like automatic restart, automatic scaling, Delta Lake consistency guarantees, and Unity Catalog security features for fault tolerance.

- ✓ **Design to support redundancy across the critical layers.** Redundancy in the critical architectural layers is key to maintaining workload continuity.

  For example, distribute clusters across availability zones by using diverse instance types and cluster pools and by implementing automatic node replacement policies. Reliable network design also protects against connectivity failures that can disrupt control plane reachability, data access, and communication with dependencies. Use redundant network paths, diverse private endpoint configurations, Domain Name System (DNS) failover mechanisms, and virtual network injection to achieve network resilience. Metadata resilience is important for maintaining compliance and data accessibility during service disruptions because governance failures can halt data access and compromise compliance requirements.

  For higher availability, consider using multiregion Azure Databricks deployments for geographic redundancy. This approach helps protect against regional outages and ensures business continuity (BC) during extended service disruptions. Multiregion setup is also a viable solution for disaster recovery (DR).

- ✓ **Implement scaling strategies.** Use automatic scaling to handle demand fluctuations while maintaining consistent performance. Plan for resource provisioning delays and

regional capacity limits. Balance the trade-off between scaling responsiveness and cluster startup latency during peak demand.

- ✓ **Adopt serverless compute for improved reliability.** Serverless compute options reduce operational complexity and improve reliability by shifting infrastructure management to Microsoft. This approach provides automatic scaling, built-in fault tolerance, and consistent availability without cluster life-cycle management overhead.

- ✓ **Implement comprehensive health monitoring and alerting.** Use comprehensive monitoring across all Azure Databricks components to proactively detect problems and respond before they affect availability. Implement automated escalation workflows for workspace health, cluster status, job execution patterns, and data pipeline performance.

- ✓ **Protect data by using Delta Lake reliability features.** Delta Lake provides essential data protection through ACID transactions, automatic versioning, time travel capabilities, and schema enforcement. These features prevent corruption and help you recover from data problems.

- ✓ **Set up job reliability and retry mechanisms.** Job reliability configurations establish resilient data processing by using intelligent retry policies, timeout management, and failure handling mechanisms that distinguish between transient problems and permanent errors.

- ✓ **Build data pipeline resilience and fault tolerance.** Data pipeline resilience addresses the critical reliability challenges of distributed data processing where failures can cascade throughout interconnected data systems and disrupt business analytics workflows.

  Advanced resilience strategies use Lakeflow Spark Declarative Pipelines, structured streaming checkpoints, Auto Loader rescued data capabilities, and Lakeflow Spark Declarative Pipelines quality constraints to provide automatic error handling, data quality enforcement, and graceful degradation during infrastructure disruptions.

- ✓ **Establish backup and DR procedures.** Effective DR requires you to align recovery time objectives (RTOs) with business requirements and establish automated backup processes for workspace metadata, notebook repositories, job definitions, cluster configurations, and integrated data storage systems.

If you use a secondary region for recovery, account for workspace metadata synchronization, code repository replication, and coordinated integration with dependent Azure services to maintain operational continuity across geographic boundaries.

- ✓ **Implement reliability testing and chaos engineering.** Systematic reliability testing validates that failure recovery mechanisms function correctly in real-world scenarios. Apply chaos engineering principles in your tests to identify resilience gaps before they affect production environments.

Recommendations

Expand table

Recommendation	Benefit
Set up cluster autoscaling with a minimum node count of two and a maximum node count that aligns with workspace quota limits. Set target utilization thresholds between 70% and 80% to balance cost efficiency with the ability to increase performance.	Automatic scaling uses dynamic node allocation to prevent cluster resource exhaustion while maintaining cost efficiency. Set up appropriate limits to ensure that workloads remain within service quotas. This approach helps you avoid jobs that fail because they exceed workspace capacity constraints.
Deploy Azure Databricks workspaces across multiple Azure regions for mission-critical workloads. Set up workspace replication with automated backup of source code, job definitions, and cluster configurations by using Databricks Asset Bundles and Azure DevOps or Azure Data Factory pipelines.	Multiregion deployments provide geographic redundancy that maintains data processing capabilities during regional outages or disasters. Automated workspace replication reduces RTOs from hours to minutes by ensuring consistent configurations across regions. This approach supports BCuity requirements and minimizes operational effects during extended regional service disruptions.
Establish cluster pools that have prewarmed instances by using different virtual machine (VM) sizes within the same family. Set up pool sizes to maintain 20% to 30% more overhead capacity than typical workload requirements.	Prewarmed cluster pools reduce cluster startup time from 5 to 10 minutes to less than 60 seconds, which helps your workload recover faster from node failures. Different VM sizing within pools ensures that cluster provisioning succeeds even when specific instance types face capacity constraints.
Activate Delta Lake time travel features by setting up automatic table versioning and retention policies. Set retention periods based on recovery requirements, which are typically 7 to 30 days for production tables.	Time travel capabilities provide point-in-time recovery (PITR) without requiring external backup systems or complex restore procedures. Automatic versioning protects against data corruption and accidental modifications while maintaining complete data lineage for compliance and debugging purposes. This approach eliminates the need for separate backup infrastructure while ensuring rapid recovery from data problems.

Integrate Azure Databricks with Azure Monitor by enabling diagnostic logs for cluster events, job execution, and data analytics. Set up custom metrics and alerts for cluster health, job failure rates, and resource utilization thresholds.	Centralized monitoring provides unified observability across all Azure Databricks components so that you can proactively detect problems before they affect production workloads. Custom alerting reduces mean time to resolution (MTTR) by automatically notifying teams when clusters experience performance degradation or job failures exceed acceptable thresholds.
Deploy serverless SQL warehouses for unplanned analytics and reporting workloads that require consistent availability without cluster management overhead.	Serverless infrastructure eliminates cluster provisioning delays and provides automatic scaling with built-in high availability guarantees. Microsoft manages all infrastructure patching, updates, and failure recovery, which reduces operational complexity and helps ensure consistent performance.
Set up Azure Databricks job retry policies to start exponential backoff at 30 seconds, and set the maximum retry count to 3. Set different retry strategies for transient failures versus configuration errors to avoid unnecessary resource consumption.	Intelligent retry mechanisms automatically recover from transient failures like network timeouts or temporary resource unavailability without manual intervention. Exponential backoff prevents overwhelming downstream services during outages. It also distinguishes between recoverable transient problems and permanent configuration problems. This approach reduces operational overhead and improves overall system resilience through automated failure recovery.
Implement virtual network injection for Azure Databricks workspaces to allow custom network routing and private connectivity. Set up network security groups (NSGs) and Azure Firewall rules to control traffic flow and integrate with existing enterprise networking infrastructure.	Virtual network injection provides network-level redundancy through custom routing options and eliminates dependency on default Azure networking paths. Private connectivity allows integration with on-premises networks and other Azure services while maintaining security isolation. This configuration supports multiple availability zones and custom load balancing strategies that improve network reliability.
Activate Unity Catalog with automated metastore backup and cross-region metadata synchronization. Set up external metastore locations in separate storage accounts to ensure metadata persistence during workspace failures.	Unity Catalog backup preserves governance policies and data lineage information during workspace disasters. Cross-region synchronization reduces metadata recovery time from hours to minutes while preserving centralized access control policies across all environments.

Deploy Lakeflow Spark Declarative Pipelines for production data pipelines that require automatic quality enforcement and fault tolerance. Set up pipeline restart policies and expectation handling to ensure data quality and maintain processing continuity.	Lakeflow Spark Declarative Pipelines automatically handles transient failures, data quality violations, and infrastructure problems without manual intervention. Built-in quality enforcement prevents corrupted data from propagating downstream, and automatic retry capabilities ensure pipeline completion during temporary resource constraints. This managed approach reduces operational overhead while maintaining data integrity standards.
Create automated workspace backup procedures by using Azure REST APIs or Databricks CLI to export source code, job and pipeline configurations, cluster settings, and workspace metadata. Schedule regular backups to Azure Storage accounts with cross-region replication turned on.	Comprehensive workspace backups allow complete environment restoration during disaster scenarios, which preserves all development work and operational configurations. Automated procedures eliminate human error and ensure backup consistency. Cross-region storage replication protects against regional outages. These approaches reduce RTOs and maintain BCuity for data teams and their analytical workflows.
Implement structured streaming and store checkpoint locations in highly available Storage accounts that have zone-redundant storage (ZRS). Set checkpoint intervals between 10 and 60 seconds based on throughput requirements and failure recovery objectives.	Checkpointing provides exactly-once processing guarantees and allows automatic recovery from cluster failures without data loss or duplicate processing. ZRS ensures checkpoint persistence across availability zone failures to maintain streaming job continuity during infrastructure disruptions.
Activate automatic cluster restart policies for sustained compute workloads. Set appropriate restart timeouts and maximum restart attempts. Turn on cluster termination detection and automatic job rescheduling for mission-critical data processing workflows.	Automatic restart policies ensure workload continuity during planned maintenance events and unexpected cluster failures without requiring manual intervention. Intelligent restart logic distinguishes between recoverable failures and permanent problems to prevent infinite restart loops while maintaining service availability for critical data processing tasks.
Set up instance pools with multiple VM families and sizes within the same compute category to provide allocation flexibility during capacity constraints.	Diverse instance type configurations ensure that cluster provisioning succeeds even when specific VM sizes experience regional capacity limitations. Mixed VM families within pools provide cost optimization opportunities and maintain performance for workload requirements. This approach reduces the risk of provisioning failures during peak demand periods.
Establish chaos engineering practices by deliberately introducing cluster failures, network partitions, and resource constraints	Proactive failure testing validates DR procedures and automatic recovery capabilities before production incidents occur.

in nonproduction environments. Automate failure injection by using Azure Chaos Studio to validate recovery procedures and identify resilience gaps.	Systematic chaos engineering identifies weak points in pipeline dependencies, cluster configurations, and monitoring systems that might not be apparent during normal operations. This approach builds confidence in system resilience while ensuring that recovery procedures work as designed during actual outages.

Security

The purpose of the Security pillar is to provide **confidentiality, integrity, and availability** guarantees to the workload.

The Security design principles provide a high-level design strategy for achieving those goals by applying approaches to the technical design of Azure Databricks.

Workload design checklist

Start your design strategy based on the design review checklist for Security and identify vulnerabilities and controls to improve the security posture. Extend the strategy to include more approaches as needed.

- ✓ **Review security baselines.** The Azure Databricks security baseline provides procedural guidance and resources for implementing the security recommendations specified in the Microsoft cloud security benchmark.
- ✓ **Integrate secure development life cycle (SLC).** Implement security code scanning for source code and MLflow model security validation to identify vulnerabilities early in the development life cycle.

  Use infrastructure as code (IaC) validation to enforce secure configurations of Azure Databricks resources.

  Protect the development environment by implementing secure source code management, managing credentials safely within development workflows, and integrating automated security testing into continuous integration and continuous delivery (CI/CD) pipelines that you use for data processing and machine learning model deployment.

- ✓ **Provide centralized governance.** Add traceability and auditing for data sources through Azure Databricks pipelines. Unity Catalog provides a centralized metadata catalog that supports data discovery and lineage tracking across workspaces with fine-grained access controls and validation.

  Unity Catalog can integrate with external data sources.

- ✓ **Introduce intentional resource segmentation.** Enforce segmentation at different scopes by using separate workspaces and subscriptions. Use separate segments for production, development, and sandbox environments to limit the effect of potential breaches.

  To apply segmentation, take the following actions:

  - Isolate sensitive data workloads in dedicated workspaces that have stricter access controls.
  - Use sandbox environments that have limited privileges and no production data access for exploratory work.

- ✓ **Implement secure network access.** Azure Databricks data plane resources, like Spark clusters and VMs, are deployed into subnets within Azure Virtual Network through virtual network injection. The control plane, which the Databricks platform manages, is isolated from the data plane, which prevents unauthorized access. The control plane communicates securely with the data plane to manage the workload, while all data processing remains within your network.

  Virtual network injection gives you control over configuration, routing, and security by using private networking capabilities in Azure. For example, you can use Azure Private Link to secure the connection to the control plane without using the public internet. You can use NSGs to control egress and ingress traffic between subnets and route traffic through Azure Firewall, NAT Gateway, or network virtual appliances for inspection and control. You can also peer the virtual network with your on-premises network, if needed.

- ✓ **Implement authorization and authentication mechanisms.** Consider identity and access management across both the control and data planes. The Azure Databricks runtime enforces its own security features and access controls while jobs run, which creates a layered security model. Azure Databricks components, like Unity Catalog and Spark clusters, integrate with Microsoft Entra ID, so you can manage access by using Azure role-

based access control (Azure RBAC) policies. This integration also provides enterprise authentication through single sign-on (SSO), multifactor authentication, and conditional access policies.

Know where your architecture relies on Databricks-native security and where it intersects with Microsoft Entra ID. This layered approach might require separate identity management and maintenance strategies.

- ✓ **Encrypt data at rest.** Azure Databricks integrates with Azure Key Vault to manage encryption keys. This integration supports customer-managed keys, so you can control the operation of your encryption keys, including revocation, auditing, and compliance with security policies.

- ✓ **Protect workload secrets.** To run data workflows, you typically need to store secrets like database connection strings, API keys, and other sensitive information. Azure Databricks natively supports secret scopes to store secrets within a workspace that you can securely access from source code and jobs.

  Secret scopes integrate with Key Vault, so you can reference secrets and manage them centrally. Enterprise teams typically need Key Vault-backed secret scopes for compliance, security, and policy enforcement.

- ✓ **Implement security monitoring.** Azure Databricks natively supports audit logging, like login attempts, notebook access, and changes to permissions. Use these logs to see admin activities in a workspace. Also, Unity Catalog access logs track who accesses what data, when they access it, and how they access it.

  Use Azure Databricks to view logs in Azure Monitor.

  The Databricks Security Analysis Tool (SAT) is also compatible with Azure Databricks workspaces.

Recommendations

Expand table

Recommendation	Benefit
Deploy Azure Databricks workspaces by using virtual network injection to establish network isolation and allow integration with corporate networking infrastructure. Set up custom network security groups, route tables, and subnet delegation to control traffic flow and enforce enterprise security policies.	Virtual network injection eliminates public internet exposure for cluster nodes and provides granular network control through custom routing and firewall rules. Integration with on-premises networks allows secure hybrid connectivity while maintaining compliance with corporate security standards.
Set up Microsoft Entra ID SSO integration with multifactor authentication and conditional access policies for workspace access. Turn on automatic user provisioning and group synchronization to streamline identity management and enforce enterprise authentication standards.	SSO integration eliminates password-related security risks and provides centralized identity management through enterprise authentication systems. Conditional access policies add context-aware security controls that evaluate user location, device compliance, and risk factors before they grant workspace access. This layered approach significantly reduces authentication-related security vulnerabilities and improves user experience.
Deploy Unity Catalog with centralized metastore configuration to establish unified data governance across all Azure Databricks workspaces. Set up hierarchical permission structures by using catalogs, schemas, and table-level access controls that have regular permission audits.	Unity Catalog provides centralized data governance that eliminates inconsistent access controls and reduces security gaps across multiple workspaces. Fine-grained permissions allow least-privilege access while audit logging supports compliance requirements and security investigations.
Activate customer-managed keys for workspace storage encryption by using Key Vault integration with automatic key rotation policies. Set up separate encryption keys for different environments and implement appropriate access controls for key management operations.	Customer-managed keys provide complete control over encryption key life cycle management and support regulatory compliance requirements for data sovereignty. Key separation across environments reduces security exposure. Automatic rotation policies maintain cryptographic hygiene without operational overhead. This approach helps you meet stringent compliance requirements like FIPS 140-2 Level 3 or Common Criteria standards.
Establish Key Vault-backed secret scopes for centralized credential management with RBAC.	Key Vault integration centralizes secrets management and provides enterprise-grade security controls like access logging and

Implement secret rotation policies and avoid storing credentials in source code or cluster configurations.	automatic rotation capabilities. This approach eliminates credential exposure in code and configuration files while enabling secure access to external systems and databases.
Create IP access lists that have allow-only policies for trusted corporate networks and deny rules for known threat sources. Set up different access policies for production and development environments based on security requirements.	IP address-based access controls provide an extra security layer that prevents unauthorized access from untrusted networks, which reduces the attack surface. Environment-specific policies enforce appropriate security levels while supporting compliance requirements for network-based access restrictions.
Set up all clusters to use secure cluster connectivity so that public IP addresses can't access them and turn off Secure Shell (SSH) access to cluster nodes. Implement cluster access modes and runtime security features to prevent unauthorized code execution.	Secure cluster connectivity eliminates public internet exposure for compute nodes while preventing direct SSH access that might compromise cluster security. Runtime security features provide extra protection against malicious code execution and lateral movement attacks within the cluster environment.
Deploy Private Link endpoints for control plane access to eliminate public internet transit for workspace connectivity. Set up private DNS zones and ensure correct network routing for seamless private connectivity integration.	Private Link eliminates public internet exposure for workspace access and ensures that all management traffic remains within the backbone network in Azure. Private connectivity provides enhanced security for sensitive workloads and supports compliance requirements that mandate private network access. This configuration reduces exposure to internet-based threats while maintaining full workspace functionality.
Activate the enhanced security and compliance settings for regulated environments that require Health Insurance Portability and Accountability Act (HIPAA), Payment Card Industry Data Security Standard (PCI DSS), or Systems and Organization Controls 2 (SOC 2) compliance. Set up automatic security updates and turn on compliance security profiles for specific regulatory frameworks.	Enhanced security and compliance features provide specialized security controls, including compliance security profiles, automatic security updates, and enhanced monitoring capabilities. This managed approach ensures continuous compliance with regulatory requirements while reducing operational overhead for security management. Automatic updates maintain security posture without disrupting business operations or requiring manual intervention.
Turn on audit logging by using Unity Catalog system tables and workspace audit logs with automated	Audit logging provides visibility into user activities, data access patterns, and system

analysis and alerting. Set up log retention policies and integrate with Security Information and Event Management (SIEM) systems for centralized security monitoring and incident response.	changes for security monitoring and compliance reporting. Integration with SIEM systems allows automated threat detection and rapid incident response capabilities through centralized log analysis.
Set up OAuth 2.0 machine-to-machine authentication for API access and automated workloads instead of personal access tokens (PATs). Implement appropriate token scoping and life cycle management to ensure secure programmatic access.	OAuth authentication provides enhanced security through fine-grained permission scoping and improved token life cycle management compared to PATs. This approach allows secure automation while maintaining appropriate access controls and audit trails for programmatic workspace interactions.
Implement workspace isolation strategies by deploying separate workspaces for different environments and establishing network segmentation controls. Set up environment-specific access policies and data boundaries to prevent cross-environment data access.	Workspace isolation prevents data leakage between environments and supports compliance requirements for data segregation and access controls. This architecture reduces the effects of security incidents and enforces environment-specific security policies that match risk profiles.
Deploy the SAT for continuous security configuration assessments that provide automated remediation recommendations. Schedule regular security scans and integrate the scans' discoveries into CI/CD pipelines for proactive security management.	Automated security assessment provides continuous monitoring of workspace configurations against security best practices and compliance requirements. Integration with development workflows applies shift-left security practices that identify and address misconfigurations before they reach production environments. This proactive approach reduces security risks while minimizing remediation costs and operational disruption.
Set up service principal authentication for automated workflows and CI/CD pipelines that have minimal required permissions. Implement credential management through Key Vault and turn on certificate-based authentication for enhanced security.	Service principal authentication eliminates dependencies on user credentials for automated processes while providing appropriate access controls and audit trails. Certificate-based authentication enhances security compared to client secrets while supporting appropriate credential life cycle management for production automation scenarios.
Establish network egress controls through virtual network injection by using custom route tables and	Network egress controls prevent unauthorized data exfiltration while providing visibility into

network security groups to monitor and restrict data transfer. Set up Azure Firewall or network virtual appliances to inspect and control outbound traffic patterns.	data movement patterns through traffic monitoring and analysis. Custom routing and firewall inspection detect unusual data transfer activities that indicate security breaches or insider threats.
Activate Microsoft Entra ID credential passthrough for Azure Data Lake Storage access to eliminate service principal dependencies. Set up user-specific access controls and ensure appropriate permission inheritance from Unity Catalog governance policies.	Credential passthrough simplifies service principal management for data access and integrates with enterprise identity systems. User-specific access controls ensure that data access permissions align with organizational policies and job functions. This approach simplifies credential management while maintaining strong security controls and audit capabilities for data lake tasks.
Implement cluster hardening practices, including SSH restriction, custom image scanning, and runtime security controls. Use approved base images and prevent unauthorized software installation by using cluster policies and init scripts validation.	Cluster hardening uses SSH restrictions to reduce attack surfaces and prevents unauthorized software installation that might compromise cluster security. Custom image scanning ensures that base images meet security standards, and runtime controls block malicious code and lateral movement within the cluster environment.
Implement automated security scanning for source code and code artifacts through CI/CD pipeline integration with static analysis tools and vulnerability scanners.	Automated security scanning helps you detect security vulnerabilities in analytical code and infrastructure configurations before they reach production environments.

Cost Optimization

Cost Optimization focuses on **detecting spend patterns, prioritizing investments in critical areas, and optimizing in others** to meet the organization's budget while meeting business requirements.

The Cost Optimization design principles provide a high-level design strategy for achieving those goals and making trade-offs as necessary in the technical design related to Azure Databricks and its environment.

Workload design checklist

Start your design strategy based on the design review checklist for Cost Optimization. Fine-tune the design so that the workload aligns with the budget that's allocated for the workload. Your design should use the right Azure capabilities, monitor investments, and find opportunities to optimize over time. Define policies and procedures to continuously monitor and optimize costs while meeting your performance requirements.

- ✓ **Determine your cost drivers.** Theoretical capacity planning often leads to overprovisioning and wasted spend. Not investing in enough resources is also risky.

  Estimate costs and seek optimization opportunities based on workload behavior. Run pilot workloads, benchmark cluster performance, and analyze automatic scaling behavior. Real usage data can help you rightsize the cluster, set scaling rules, and allocate the right resources.

- ✓ **Set clear accountability for spend.** When you use multiple Azure Databricks workspaces, track which teams or projects are responsible for specific costs. This task requires tagging resources, like clusters or jobs, with project or cost center information, using chargeback models to assign usage-based costs to teams, and setting budget controls to monitor and limit spending.

- ✓ **Choose the appropriate tiers.** We recommend that you use the Standard tier for development and basic production workloads. Use the Premium tier for production workloads because it provides security features, like the Unity Catalog, that are essential for most analytics workloads.

- ✓ **Choose between serverless compute versus VMs.** Serverless compute uses consumption-based pricing, so you only pay for what you use. We recommend that you use serverless compute for workloads that have activity spikes or on-demand jobs because it scales automatically and reduces operational overhead. You don't need to manage infrastructure or pay for idle time.

  For predictable or steady usage, choose VM-based clusters. This approach gives you more control but requires operational management and tuning to avoid overprovisioning. If you're sure about long-term usage, use reserved capacity. Databricks Commit Units (DBCU) are prepaid usage contracts that give discounts in exchange for usage commitments.

Make sure that you analyze historical trends and project future demands to make the best choice.

- ✓ **Optimize cluster utilization.** Reduce Azure Databricks costs by automatically scaling and shutting down clusters when you don't need them.

  Evaluate whether your budget allows for cluster pools. Cluster pools can reduce cluster start times, but they're idle resources that accrue infrastructure costs while you're not using them.

  Save costs in development and test environments by using scaled down configurations. Encourage cluster sharing among teams to avoid using unnecessary resources. Enforce automatic termination policies to deprovision idle clusters.

- ✓ **Optimize compute for each workload.** Different workloads require different compute configurations. Some jobs might need higher memory and processing power, while other workloads might run lightweight jobs that accrue lower costs.

  Instead of using the same large cluster for every job, assign the right cluster to each job. Use Azure Databricks to tailor compute resources to match each workload. This approach helps you reduce costs and improve performance.

- ✓ **Optimize storage costs.** Storing large volumes of data can get expensive. Reduce costs by using Delta Lake capabilities. For example, use data compaction to merge many small files into fewer large files to reduce storage overhead and speed up queries.

  Diligently manage old data. You can use retention policies to remove outdated versions. You can also move old, infrequently accessed data to cheaper storage tiers. If applicable, automated life cycle policies, like time-based deletion or tiering rules, help archive or delete data when it becomes less useful.

  Different storage formats and compression settings can also reduce the amount of space that you use.

- ✓ **Optimize data processing techniques.** Compute, networking, and querying when you process large volumes of data accrue costs. To reduce these costs, use a combination of strategies for query tuning, data format selection, and Delta Lake and code optimizations.

- *Minimize data movement.* Evaluate the data processing pipeline to reduce unnecessary data movement and bandwidth costs. Implement incremental processing to avoid reprocessing unchanged data, and use caching to store frequently accessed data closer to compute resources. Reduce overhead when connectors access or integrate with external data sources.

- *Use efficient file formats.* Formats like Parquet and compression algorithms like Zstandard that are native to Databricks lead to faster read times and lower data costs because less data needs to move.

- *Make your queries efficient.* Avoid full-table scans to reduce compute costs. Instead, partition your Delta tables based on common filter columns. Use native features to reduce compute time. For example, native Spark features like Catalyst optimzer and adaptive query execution (AQE) dynamically optimize joins and partitioning at runtime. Databricks Photon engine runs queries faster.

- *Apply code optimization design patterns.* Use patterns like Competing Consumers, Queue-Based Load Leveling, and Compute Resource Consolidation within Azure Databricks environments.

✓ **Monitor consumption.** Databricks Unit (DBU) is an abstracted billing model based on compute usage. Azure Databricks gives you detailed information that provides visibility into usage metrics about clusters, runtime hours, and other components. Use that data for budget planning and cost control.

✓ **Implement automated spending guardrails.** To avoid overspending and ensure efficient use of resources, enforce policies that regulate resource usage. For example, have checks on the types of clusters that can be created, and limit the cluster size or its lifetime. Set alerts to notify you when resource usage approaches the allowed budget boundaries. For example, if a job suddenly starts to consume a specific number of DBUs, a script can alert the admin or shut down the job.

Take advantage of Databricks system tables to track cluster usage and DBU consumption. You can query the table to detect cost anomalies.

Recommendations

Expand table

Recommendation	Benefit
Deploy job clusters for scheduled workloads instead of all-purpose clusters to eliminate idle compute costs. Set jobs to automatically end when they finish.	Job clusters reduce costs by automatically ending jobs when they finish and optimizing DBU consumption by precisely matching compute time to actual processing requirements.
Turn on cluster autoscaling and set minimum and maximum node limits based on workload analysis to handle baseline loads and peak demand requirements. Set up scaling policies to respond quickly to workload changes and avoid unnecessary scaling oscillations that can increase costs unnecessarily.	Automatic scaling further reduces overprovisioning costs compared to fixed-size clusters. It maintains performance levels during peak periods and automatically reduces resources during low-demand periods.
Set up automatic termination for all interactive clusters with appropriate timeout periods based on usage patterns. These periods are typically 30 to 60 minutes for development environments.	Automatic termination reduces interactive cluster costs without affecting user productivity. This approach eliminates costs from clusters that run overnight or on weekends.
Adopt serverless SQL warehouses for interactive SQL workloads to eliminate infrastructure management overhead and optimize costs through consumption-based billing. Set up appropriate sizing based on concurrency requirements and turn on autostop functionality to minimize costs during inactive periods. Migrate from classic SQL endpoints to serverless SQL warehouses for better performance and cost efficiency. Use built-in Photon acceleration capabilities.	Serverless SQL warehouses further reduce SQL workload expenses compared to always-on clusters by applying usage-based billing that eliminates idle time costs. Built-in Photon acceleration improves performance while providing predictable costs for each query for interactive analytics scenarios.
Implement cluster pools for frequently used configurations to reduce startup times and optimize resource allocation based on usage patterns and demand forecasting.	Cluster pools reduce startup time from minutes to seconds while eliminating DBU charges for idle pool instances.
Use Delta Lake optimization features, including `OPTIMIZE` commands, `Z-ORDER` clustering, and `VACUUM` operations, to reduce storage costs and improve query performance.	Delta Lake optimization reduces storage costs through data compaction and efficient compression. It improves query performance by reducing file scan requirements.

Schedule regular optimization jobs to compact small files, implement data retention policies, and set compression settings based on data access patterns.	
Implement compute policies to enforce cost-effective configurations across all workspaces and teams by restricting instance types and enforcing automatic termination settings. Create different policy templates for development, staging, and production environments that have different levels of restrictions and appropriate tags for cost attribution.	Compute policies reduce average cluster costs by preventing overprovisioning and ensuring adherence to cost optimization standards while maintaining governance.
Monitor costs by using Databricks system tables and Microsoft Cost Management integration to see DBU consumption patterns and spending trends. Implement automated cost reporting dashboards that track usage by workspace, user, job, and cluster type. Set up cost alerts for proactive management. Use Unity Catalog system tables to analyze detailed usage patterns and create chargeback models for different teams and projects based on actual resource consumption.	Use comprehensive cost monitoring to see DBU consumption patterns and accurately attribute costs through detailed usage analytics and tagging strategies. Integrate with Cost Management to use organization-wide cost governance and establish responsible resource usage patterns across teams.
Purchase Databricks reserved capacity through Databricks Commit Units (DBCU) for predictable workloads that have stable usage patterns and optimal commitment terms.	Reserved capacity achieves more cost savings through DBCU compared to pay-as-you-go pricing while providing cost predictability over one to three-year terms for stable production workloads.
Optimize workload-specific compute configurations by selecting appropriate compute types for different use cases, like job clusters for extract, transform, load (ETL) pipelines and graphics processing unit (GPU) instances for machine learning training. Match instance types and cluster configurations to specific workload requirements rather than using generic configurations across all scenarios.	Workload-specific optimization further reduces costs compared to one-size-fits-all approaches by eliminating overprovisioning and using specialized compute types optimized for specific use cases.

Implement automated data life cycle policies with scheduled cleanup operations, including VACUUM commands, log file retention, and checkpoint management, based on business requirements.	Automated life cycle management reduces storage costs by systematically removing unnecessary data versions, logs, and temporary files and preventing storage bloat over time.
Use the Standard tier for development and testing environments. Use the Premium tier only for production workloads that require advanced security features and compliance certifications.	Strategic tier selection optimizes licensing costs by using the Standard tier for nonproduction workloads where advanced security features aren't required. Premium tier features like RBAC and audit logging are applied only where business requirements and security policies justify the extra cost investment.
Implement serverless jobs for variable and intermittent workloads that have unpredictable scheduling patterns or resource requirements for unplanned analytics and experimental workloads. Set up serverless compute for batch processing jobs where usage patterns are difficult to predict and use automatic optimization capabilities. Migrate appropriate workloads from traditional clusters to serverless compute based on usage analysis and cost-benefit evaluation to optimize resource utilization.	Serverless jobs eliminate idle time costs and provide automatic optimization for variable resource requirements, which reduces costs for unpredictable workloads. The consumption-based billing model ensures that you pay only for compute time that you use, which makes it ideal for development environments and sporadic production workloads that need automatic resource optimization.
Set up cost alerts and budgets by using Cost Management and Databricks usage monitoring to proactively manage costs with multiple alert thresholds. Set up escalation procedures for different stakeholder groups and implement automated responses for critical cost overruns. Review budgets regularly.	Proactive cost monitoring helps you detect cost anomalies and budget overruns early so that you can prevent surprise expenses and act before costs significantly affect budgets.
Optimize data formats and turn on Photon acceleration to reduce compute time through efficient data processing with columnar storage formats and compression algorithms. Implement partitioning strategies that minimize data scanning requirements and use Photon	Data format optimization and Photon acceleration reduce compute time and costs through columnar storage optimizations and vectorized query execution capabilities. These optimizations compound over time as data volumes grow, providing increasing cost benefits for

acceleration for supported workloads to take advantage of vectorized query execution.	analytical workloads and complex data processing pipelines without requiring architectural changes.

Lakeflow Connect Free Tier provides 100 free Databricks Units (DBUs) per workspace per day for data ingestion from SaaS applications and databases. This free tier lowers the entry point for Lakehouse adoption, enabling teams to validate ingestion pipelines and Unity Catalog governance before scaling to production volumes. Factor in the 100 DBU/day limit when sizing initial deployments because standard pricing applies beyond this threshold.

Operational Excellence

Operational Excellence primarily focuses on procedures for **development practices, observability, and release management**.

The Operational Excellence design principles provide a high-level design strategy for achieving those goals for the operational requirements of the workload.

Workload design checklist

Start your design strategy based on the design review checklist for Operational Excellence for defining processes for observability, testing, and deployment related to Azure Databricks.

- ✓ **Collect monitoring data.** For your Azure Databricks workload, focus on tracking key areas like cluster health, resource usage, jobs and pipelines, data quality, and access activity. Use these metrics to confirm that the system behaves as expected. You can also use them to audit how data and resources are accessed and used and to enforce governance.
  - *Monitor the cluster.* When you monitor Azure Databricks clusters, focus on indicators that reflect performance and efficiency. Track overall cluster health and observe how nodes use resources like central processing unit (CPU), memory, and disks.
  - *Monitor jobs and pipelines.* Capture metrics that show how jobs flow when they run. These metrics include job success and failure rates and run durations. Gather information about how jobs are triggered to learn why they run.

    Use Databricks System tables to capture job status, dependency chains, and throughput natively.

- *Monitor data source connectivity.* Monitor integrations and dependencies with external systems. This data includes source connectivity status, API dependencies, and service principal authentication behavior. You can use Unity Catalog to manage and monitor external locations. This approach helps you identify potential access or configuration problems.

- *Monitor data quality.* Collect signals that validate both the integrity and freshness of your data. Monitor for schema evolution problems by using tools like Auto Loader. Implement rules that do completeness checks, null value detection, and anomaly identification. You can use Lakeflow Spark Declarative Pipelines to enforce built-in quality constraints during data processing.

  Capturing data lineage through Unity Catalog helps you trace how data flows and transforms across systems, which gives your pipelines greater transparency and accountability.

Built-in monitoring tools in Azure Databricks integrate with Azure Monitor.

✓ **Set up automated and repeatable deployment assets.** Use IaC to define and manage Azure Databricks resources.

Automate provisioning of workspaces, including region selection, networking, and access control, to ensure consistency across environments. Use cluster templates to standardize compute configurations. This approach reduces the risk of misconfiguration and improves cost predictability. Define jobs and pipelines as code by using formats like JSON Azure Resource Manager templates (ARM templates) so that they're version-controlled and reproducible.

Use branching strategies and rollback procedures in Databricks Asset Bundles to version control notebook source code, job configurations, pipeline definitions, and infrastructure settings in Git repositories.

✓ **Automate deployments.** Use CI/CD pipelines in Azure Databricks to automate the deployment of pipelines, job configurations, cluster settings, and Unity Catalog assets. Instead of manually pushing changes, consider tools like Databricks Repos for version control, Azure DevOps or GitHub Actions for pipeline automation, and Databricks Asset Bundles for packaging code and configurations.

- **Automate routine tasks.** Commonly automated tasks include managing cluster life cycles, like scheduled starts and stops, cleaning up logs, and validating pipeline health. By integrating with Azure tools like Azure Logic Apps or Azure Functions, teams can build self-healing workflows that automatically respond to problems, like restarting failed jobs or scaling clusters. This type of automation helps maintain reliable, efficient Azure Databricks operations when workloads grow.

- **Have strong testing practices.** Azure Databricks-specific strategies include unit testing for notebook code, integration testing for data pipelines, validation of Lakeflow Spark Declarative Pipelines logic, permission testing with Unity Catalog, and verifying infrastructure deployments. These practices help catch problems early and reduce incidents in production.

- **Develop operational runbooks to handle incidents.** Operational runbooks provide structured, step-by-step guidance for handling common Azure Databricks scenarios. These runbooks include diagnostic commands, log locations, escalation contacts, and recovery procedures with estimated resolution times. Use runbooks to quickly and consistently respond to incidents across teams.

- **Develop backup and recovery procedures.** Backup and recovery procedures ensure BCuity through protection of workspace configurations, analytics source code, job definitions, and data assets. Backup and recovery procedures include automated backup schedules and cross-region replication that meet RTOs and recovery point objectives (RPOs).

- **Implement team collaboration and knowledge management.** Team collaboration practices optimize Azure Databricks productivity through shared workspace organization, notebook collaboration features, and documentation standards that facilitate knowledge transfer and reduce project duplication across development teams.

- **Use serverless workspaces for rapid environment provisioning.** Serverless workspaces provide fully managed workspace types with preconfigured serverless compute, default DBFS storage, and other benefits. These workspaces enable workspace creation from Azure portal, automatic Unity Catalog enablement, and native integration with Entra ID for all tenant users. This gives the ease of instantly creating workspaces without provisioning compute or storage, reducing manual operations.

Recommendations

Expand table

Recommendation	Benefit
Set up diagnostic settings for Azure Databricks workspaces to send platform logs, audit logs, and cluster events to an Azure Monitor Log Analytics workspace. Turn on all available log categories, including workspace, clusters, accounts, jobs, notebook, and Unity Catalog audit logs, for observability coverage.	Centralizes all Azure Databricks telemetry in Log Analytics and turns on advanced KQL queries for troubleshooting, automated alerting on critical events, and compliance reporting. Diagnostics provide unified visibility across workspace activities, cluster performance, and data access patterns for proactive operational management.
Deploy Azure Databricks workspaces by using ARM templates or Bicep files that have parameterized configurations for consistent environment provisioning. Include workspace settings, network configurations, Unity Catalog enablement, and security policies in the template definitions to ensure standardized deployments across development, testing, and production environments.	Eliminates configuration drift between environments and reduces deployment errors through consistent, version-controlled infrastructure definitions. Further accelerates environment provisioning compared to manual deployment processes and allows rapid recovery through automated workspace re-creation during disaster scenarios.
Integrate Azure Databricks notebooks and other source code with Git repositories by using Databricks Repos for source control and collaborative development. Set up automated CI/CD pipelines through Azure DevOps or GitHub Actions to deploy source code changes, job and pipeline configurations, and cluster templates across environments with appropriate testing and approval workflows.	Allows collaborative development with version history, branch-based workflows, and merge conflict resolution for code. Reduces deployment risks through automated testing and staged releases while maintaining complete audit trails of all production changes.
Deploy automated cluster rightsizing solutions by using Azure Databricks cluster metrics and Azure Monitor data to analyze utilization patterns and recommend optimal instance types and sizes. Set up automatic scaling policies based on CPU, memory, and job queue metrics to automatically adjust cluster capacity according to workload demands.	Optimizes infrastructure costs by automatically matching cluster resources to actual workload requirements. Maintains performance service-level objectives (SLAs) and reduces compute costs through intelligent resource allocation and automated scaling decisions. Eliminates manual monitoring overhead and allows proactive capacity management through data-driven

	insights about resource usage patterns and optimization opportunities.
Activate Unity Catalog audit logging to track all data access tasks, permission changes, and governance activities within Azure Databricks workspaces. Set up log retention policies and integrate with Microsoft Sentinel or partner SIEM solutions for automated security monitoring and compliance reporting.	Provides complete audit trails for data access patterns, permission modifications, and governance tasks required for regulatory compliance frameworks. Allows automated threat detection and investigation of suspicious data access behaviors through centralized security monitoring.
Implement Lakeflow Spark Declarative Pipelines that have data quality expectations and monitoring rules to automate data validation and pipeline quality assurance. Set up expectation thresholds, quarantine policies, and automated alerting for data quality violations to maintain pipeline reliability and data integrity.	Automates data quality validation by using declarative rules that prevent bad data from propagating downstream, which reduces manual validation efforts. Provides transparent data quality metrics and automated remediation workflows that maintain pipeline reliability and business confidence in data accuracy.
Establish automated backup procedures for Azure Databricks workspace artifacts by using the Databricks REST API and Azure Automation runbooks. Schedule regular backups of analytics source content, job definitions, cluster configurations, and workspace settings with versioned storage in Storage accounts and cross-region replication.	Ensures rapid recovery from accidental deletions, configuration changes, or workspace corruption by using automated restoration capabilities. Maintains BCuity through versioned backups and reduces RTOs from days to hours through standardized backup and restore procedures.
Create standardized workspace folder hierarchies by using naming conventions that include project codes, environment indicators, and team ownership. Implement shared folders for common libraries, templates, and documentation with appropriate access controls to facilitate knowledge sharing and collaboration.	Improves project discoverability and reduces onboarding time for new team members through consistent workspace organization. Accelerates development through shared code libraries and standardized project structures that eliminate duplication of effort across teams.
Set up Cost Management with resource tagging strategies for Azure Databricks workspaces, clusters, and compute resources.	Provides granular cost visibility and accountability across organizational units through detailed spend analysis and automated budget monitoring. Proactively

Implement cost alerts, budget thresholds, and automated reporting to track spending across projects, teams, and environments with chargeback capabilities and optimization recommendations.	optimize costs through spending alerts and usage pattern insights that prevent budget overruns and identify optimization opportunities. Supports accurate cost allocation and chargeback processes with detailed resource utilization reporting and automated cost center assignment based on resource tags.
Set up service principal authentication for Azure Databricks integrations with external systems, data sources, and Azure services. Implement managed identity where possible and establish credential rotation policies with Key Vault integration for secure, automated authentication management.	Eliminates shared credential security risks and allows automated authentication without manual intervention. Provides centralized credential management with audit trails and supports fine-grained access control policies that align with least-privilege security principles.
Establish cluster life cycle policies that have automated termination schedules, idle timeout configurations, and resource usage limits to enforce organizational governance standards. Set up policy-based cluster creation restrictions, instance type limitations, and maximum runtime controls to prevent resource waste and ensure compliance.	Reduces compute costs through automated cluster life cycle management and prevents resource waste from idle or forgotten clusters. Enforces organizational policies consistently across all users and teams while maintaining operational flexibility for legitimate use cases.
Deploy Azure Monitor alert rules for critical Azure Databricks tasks, including cluster failures, job execution errors, workspace capacity limits, and Unity Catalog access violations. Set up automated notification workflows that have escalation procedures and integrate with incident management systems like ServiceNow or Jira.	Helps you proactively respond to incidents by notifying you of critical problems before they affect business operations. Reduces mean time to detection (MTTD) from hours to minutes and supports automated escalation procedures that notify the right team members based on severity levels.
Implement environment-specific workspace configurations with RBAC policies that enforce separation between development, testing, and production environments. Set up Unity Catalog governance rules, network security groups, and data access	Prevents unauthorized access to production data and reduces risk of accidental changes in critical environments through enforced security boundaries. Maintains regulatory compliance by ensuring that development activities can't affect production systems

permissions that meet each environment's security and compliance requirements.	and data integrity is preserved across environment boundaries.

Performance Efficiency

Performance Efficiency is about **maintaining user experience even when there's an increase in load** by managing capacity. The strategy includes scaling resources, identifying and optimizing potential bottlenecks, and optimizing for peak performance.

The Performance Efficiency design principles provide a high-level design strategy for achieving those capacity goals against the expected usage.

Workload design checklist

Start your design strategy based on the design review checklist for Performance Efficiency. Define a baseline that's based on key performance indicators for Azure Databricks.

- ✓ **Plan capacity.** Analyze workloads and monitor resource usage to determine how much compute and storage your workloads actually need. Use that information to rightsize clusters, optimize job schedules, and forecast storage growth. This approach helps you avoid underprovisioning, which leads to resource constraints.

- ✓ **Choose optimal compute configurations for workload characteristics.** Evaluate serverless options, which can provide better automatic scaling and faster startup times. Compare them with traditional clusters to choose the best fit.

  For clusters, optimize configurations, including instance types, sizes, and scaling settings, based on data volume and processing patterns. Be sure to analyze trade-offs between instance families for specific use cases. For example, evaluate memory-optimized versus compute-optimized instances and local solid-state drives (SSDs) versus standard storage options to match performance requirements.

  Spark clusters can run different types of workloads, which require their unique performance tuning. In general, you need to run jobs faster and avoid compute bottlenecks. Fine-tune settings like executor memory, parallelism, and garbage collection to achieve these goals.

For more information about how to choose the right services for your workload, see Architecture strategies for selecting the right services.

- ✓ **Prioritize resource allocation for critical workloads.** Separate and prioritize workloads that run at the same time. Use features like resource pools, cluster pools, isolation modes, and job queues to avoid interference between jobs. Set resource quotas and scheduling rules to help ensure that background or lower-priority processes don't slow down high-priority tasks.

- ✓ **Set up automatic scaling for variable workloads.** Set up automatic scaling policies in Azure Databricks by defining scaling triggers that cause the cluster to scale, determine how quickly it adds or removes nodes, and set resource limits. These settings help Azure Databricks respond efficiently to changing workloads, optimize resource usage, and avoid performance problems during scaling events.

- ✓ **Design efficient data storage and retrieval mechanisms.** Performance improvements for data-intensive tasks require careful planning and tuning.

  - *Organize data strategically.* Design data partitioning schemes that optimize query performance when you organize Delta Lake tables. Good partitioning allows Spark to prune partitions so that it reads only the relevant subsets of data during a query instead of scanning the entire table.

    File sizing plays a key role. Files that are too small create excessive metadata overhead and slow down Spark jobs, while files that are too large can cause memory and performance problems.

    Align your data layout with how users or jobs typically query the data. Otherwise, full-table scans might degrade performance.

  - *Implement effective caching.* Use caching for hot datasets and monitor cache hit ratios to ensure that you aren't unnecessarily using memory. Spark provides built-in caching mechanisms and Azure Databricks provides Delta Cache, which further optimizes by caching data at the disk level across nodes.

  - *Write efficient queries.* Avoid unnecessary data scans, excessive shuffling, and long run times, which all contribute to inefficient performance.

Use indexing, predicate pushdown, projection pushdown, and join optimization techniques that use query plan analysis to run jobs more efficiently to optimize SQL queries and Spark operations.

Azure Databricks provides built-in optimizations. The Catalyst optimizer rewrites queries for efficiency. AQE adjusts plans at runtime to handle data skew and improve joins. Delta Lake features like table statistics, Z-order clustering, and Bloom filters further reduce scanned data for faster, more cost-effective queries.

- *Choose the right data formats and compression.* Formats like Parquet and smart compression algorithms like Zstandard (zstd) reduce storage and speed up reads without compromising performance.

✓ **Optimize network and input/output (I/O) performance.** Choose high-performance storage options like Premium or SSD-backed storage and design your architecture to minimize data movement by processing data close to where it's stored.

Also use efficient data transfer strategies, like batching writes and avoiding unnecessary shuffles, to maximize throughput and reduce latency.

✓ **Optimize job execution based on the type of workload.** Tailor optimization strategies to specific needs.

- *Stream processing:* Real-time data pipelines require low-latency and high-throughput performance. In Azure Databricks, you must tune parameters like trigger intervals, micro-batch sizes, watermarking, and checkpointing to meet these requirements. Use Structured Streaming and Delta Lake capabilities like schema evolution and exactly-once delivery to ensure consistent processing under different loads.

- *Machine learning:* Machine learning training and inference jobs are typically compute-intensive. You can boost performance by using distributed training, GPU acceleration, and efficient feature engineering pipelines. Azure Databricks supports machine learning performance tuning through MLflow, Databricks Runtime for machine learning, and integrations with tools like Horovod. Tune resource configurations and apply data preprocessing optimizations to significantly reduce training time and inference latency.

Use Lakeflow Spark Declarative Pipelines to simplify and automate the implementation of these optimization recommendations.

- ✓ **Use your monitoring system to identify performance bottlenecks.** Implement comprehensive performance monitoring to learn how jobs, clusters, and queries behave so that you can identify bottlenecks or inefficiencies that increase costs and slow down workloads.

  Analyze anomalies in key metrics like CPU and memory usage, job run times, query latencies, and cluster health. This information helps you pinpoint slowdowns that poor Spark configurations, unoptimized queries, or underprovisioned and overprovisioned clusters can cause.

  Use built-in tools like the Spark UI to analyze query plans and job stages, Azure Monitor to track infrastructure-level metrics, and custom metrics or logs for deeper insights. These tools support proactive tuning so that you can fix problems before they affect users or critical pipelines.

- ✓ **Conduct systematic performance testing.** Use load testing, stress testing, and benchmarking to validate run times, resource usage, and system responsiveness. Establish performance baselines and incorporate automated tests into your CI/CD pipelines to detect slowdowns early and measure the effects of optimizations.

Recommendations

Expand table

Recommendation	Benefit
Set up Azure Databricks clusters to use memory-optimized instance types like E-series or M-series VMs when you process large datasets that require extensive in-memory caching, machine learning model training, or complex analytical tasks. Evaluate memory requirements based on dataset size and processing patterns, and then select VM sizes that provide sufficient memory capacity and high memory-to-CPU ratios for optimal performance.	Eliminates memory bottlenecks that can cause job failures or severe performance degradation. Helps memory-intensive operations like large-scale machine learning training and complex analytics workloads run smoothly.
Set up cluster autoscaling policies to use appropriate minimum and maximum node limits based on workload patterns and performance requirements. Set minimum nodes to handle baseline workloads efficiently while establishing maximum limits to prevent runaway costs. Define scaling triggers based on CPU utilization, memory usage, or job queue depth, and configure scaling velocity to balance responsiveness with cost optimization.	Maintains consistent performance during demand fluctuations while optimizing costs through automatic resource adjustment that scales up during peak periods and scales down during low utilization periods.
Run OPTIMIZE commands with Z-ordering on Delta Lake tables to improve data clustering and query performance. Choose Z-order columns based on frequently used filter and join conditions in your queries. These conditions typically include columns used in `WHERE` clauses, `GROUP BY` operations, and `JOIN` predicates. Schedule regular optimization tasks by using Azure Databricks jobs or Lakeflow Spark Declarative Pipelines to maintain optimal performance as data grows.	Reduces query run time through improved data skipping and minimized I/O operations, while also decreasing storage costs through better compression ratios achieved by clustering related data together. Provides cumulative performance improvements as optimization benefits compound over time with regular maintenance and intelligent data organization that aligns with actual query patterns.
Turn on Delta Cache for cluster configurations where you frequently access the same datasets across multiple queries or jobs. Edit cache settings to use local nonvolatile memory express (NVMe) SSD storage effectively to allocate adequate cache size based on your dataset characteristics and access patterns. Monitor cache hit ratios and adjust cache configurations to maximize performance benefits for your specific workloads.	Accelerates query performance for frequently accessed data through intelligent SSD-based caching that bypasses slower network storage. This approach significantly reduces latency for iterative analytics and machine learning workloads.

Turn on Photon engine for cluster configurations and SQL warehouses to accelerate SQL queries and DataFrame operations through vectorized execution. Photon provides the most significant benefits for analytical workloads that have aggregations, joins, and complex SQL tasks. Set up Photon-enabled compute resources for data engineering pipelines, business intelligence (BI) workloads, and analytical applications that process large datasets.	Improves performance for SQL and DataFrame operations through native vectorized execution. Reduces compute costs by improving processing efficiency and reducing run time. Allows you to process larger datasets within the same time constraints and supports more concurrent users without degrading performance by improving overall system throughput.
Set Spark executor memory settings between 2 and 8 gigabytes (GB) for each executor and driver memory based on your largest dataset size and processing complexity. Set `spark.executor.cores` to two to five cores for each executor to balance parallelism and resource efficiency. Adjust these settings based on your specific workload characteristics, data volume, and cluster size to prevent out-of-memory errors while maximizing resource utilization.	Prevents job failures that memory problems cause and optimizes resource allocation efficiency to reduce run time and resource waste.
Set up Storage accounts with Premium SSD performance tiers for Azure Databricks workloads that require high input/output operations per second (IOPS) and low latency. Use Premium block blob storage for data lake scenarios that have intensive read and write tasks. Ensure that storage accounts are in the same region as your Azure Databricks workspace to minimize network latency.	Provides up to 20,000 IOPS and submillisecond latency for storage operations. Improves performance for data-intensive workloads and reduces job run times by eliminating storage I/O bottlenecks.
Design data partitioning strategies based on commonly used filter columns in your queries. These filters include date columns for time-series data or categorical columns for dimensional data. Avoid overpartitioning by limiting partitions to fewer than 10,000 and ensuring that each partition has at least 1 GB of data. Use partition pruning-friendly query patterns and consider liquid clustering for tables that have multiple partition candidates.	Reduces data scanning through effective partition pruning. Improves query performance and reduces compute costs by processing only relevant data partitions. Queries behave predictably and scale linearly with filtered data size rather than total table size. Maintains consistent response times as datasets grow to petabyte (PB) scale.
Use Parquet file format with zstd or Snappy compression for analytical workloads to optimize storage efficiency and query	Reduces storage costs while improving query performance

performance. Zstd provides better compression ratios for cold data. Snappy provides faster decompression for frequently accessed datasets. Set appropriate compression levels and evaluate compression trade-offs based on your access patterns and storage costs.	through columnar storage efficiency and optimized compression. Scans data faster and reduces network I/O.
Deploy serverless SQL warehouses for BI and analytical workloads that require unplanned querying and interactive analytics. Set up appropriate warehouse sizes, like 2X-Small to 4X-Large, based on concurrency requirements and query complexity. Turn on autostop and autoresume features to optimize costs while ensuring rapid query responsiveness for users.	Eliminates cluster management overhead while providing instant scaling and Photon-accelerated performance. Delivers better price performance compared to traditional clusters for SQL workloads. Provides consistent subsecond query startup times and automatic optimization that adapts to changing workload patterns without manual intervention or configuration tuning.
Turn on AQE in Spark configurations to use runtime optimization capabilities, including dynamic coalescing of shuffle partitions, dynamic join strategy switching, and optimization of skewed joins. Set AQE parameters like target shuffle partition size and coalescing thresholds based on your typical data volumes and cluster characteristics.	Improves query performance through intelligent runtime optimizations that adapt to actual data characteristics and run patterns. Automatically addresses common performance problems like small files and data skew.
Create cluster pools that have prewarmed instances that match your most common cluster configurations to reduce startup times for interactive clusters and job clusters. Set pool sizes based on expected concurrent usage patterns and maintain idle instances during peak hours to ensure immediate availability for development teams and scheduled jobs.	Reduces cluster startup time from 5 to 10 minutes to less than 30 seconds. Improves developer productivity and runs jobs faster for time-sensitive data processing workflows.

Schedule regular optimization tasks by using Azure Databricks jobs to compact small files and improve query performance. Run `VACUUM` commands to clean up expired transaction logs and deleted files. Set optimization frequency based on data ingestion patterns, typically daily for high-volume tables and weekly for less frequently updated tables. Monitor table statistics and file counts to determine optimal maintenance schedules.	Maintains consistent query performance as data volumes grow by preventing file proliferation and data fragmentation. Reduces storage costs by cleaning up unnecessary files and improving compression ratios. Prevents performance degradation over time that commonly occurs in data lakes without appropriate maintenance. Ensures predictable query response times and optimal resource utilization.
Configure structured streaming trigger intervals based on latency requirements and data arrival patterns. Use continuous triggers for subsecond latency needs or micro-batch triggers with 1-second to 10-second intervals for balanced performance. Optimize checkpoint locations by using fast storage and configure appropriate checkpoint intervals to balance fault tolerance and performance overhead.	Achieves optimal balance between latency and throughput for real-time data processing. Allows consistent stream processing performance that can handle different data arrival rates while maintaining low end-to-end latency.
Deploy GPU-enabled clusters by using NC, ND, or NV-series VMs for deep learning model training and inference workloads. Configure appropriate GPU memory allocation and use MLflow for distributed training orchestration. Select GPU instance types based on model complexity and training dataset size. Consider both memory capacity and compute performance requirements for your specific machine learning workloads.	Accelerates model training by 10 to 100 times compared to CPU-only clusters by using parallel processing capabilities designed for machine learning operations. Reduces training time and allows faster model iteration cycles.

Azure policies

Azure provides an extensive set of built-in policies related to Azure Databricks and its dependencies. Some of the preceding recommendations can be audited through Azure Policy. For example, you can check whether:

- Azure Databricks workspaces use virtual network injection for enhanced network security and isolation.
- Azure Databricks workspaces block public network access when you use private endpoints.
- Azure Databricks clusters have disk encryption turned on to protect data at rest.

- Azure Databricks workspaces use customer-managed keys for enhanced encryption control.
- Azure Databricks workspaces have diagnostic logging turned on for monitoring and compliance.
- Azure Databricks workspaces are only deployed in approved geographic regions for compliance.
- Enterprise workloads use Azure Databricks Premium tier for enhanced security and compliance features.
- Azure Databricks workspaces use Unity Catalog for centralized data governance.

For comprehensive governance, review the Azure Policy built-in definitions for Azure Databricks and other policies that might affect the security of the analytics platform.

Azure Advisor recommendations

Azure Advisor is a personalized cloud consultant that helps you follow best practices to optimize your Azure deployments.

For more information, see Azure Advisor.

Trade-offs

You might have to make design trade-offs if you use the approaches in the pillar checklists.

Analyze performance and cost trade-offs

Balance performance and cost to get the most value from your workloads. Overprovisioning wastes money, while underprovisioning can slow down workloads or lead to failures. Test different configurations, use performance benchmarks, and analyze costs to guide your choices.

Scenario architecture

Stream processing by using Azure Databricks shows a foundational architecture that demonstrates the key recommendations described in this article.

Related content

- Azure Databricks documentation
- Introduction to the well-architected data lakehouse

Feedback

Was this page helpful? Yes No

Last updated on 02/12/2026

English (United States) Your Privacy Choices Theme

AI Disclaimer Previous Versions Blog Contribute Privacy
Consumer Health Privacy Terms of Use Trademarks © Microsoft 2026